

UNIVERSIDAD DEL NORTE (UniNorte)

Facultad de Ingeniería

Carrera: Ingeniería en Informática



AVANCE 1 — INFORME DE PLANIFICACIÓN

***“Sistema de Sincronización Canvas LMS ↔ Microsoft Teams:
caso de gestión de identidad académica en USIL Paraguay”***

Asignatura: Programación IV

Docente: Prof. Marcos Arrieta Moreira

Grupo N.º 3

N.º	Nombres y Apellidos Completos	Cédula de Identidad	Rol en el Equipo
1	Roque Esteche	C.I. N.º 6.868.066	Scrum Master / Project Manager
2	Hypatia García	C.I. N.º 5.444.430	Backend & Software Architect Lead
3	María Iglesias	C.I. N.º 7.330.668	DevOps & Cloud Engineer
4	Jesús Domínguez	C.I. N.º 5.666.290	Frontend & QA Engineer
5	Alfonzo Martínez	C.I. N.º 4.826.141	Frontend & QA Engineer

Asunción, Paraguay

2026



Índice

1. Introducción	1
2. Objetivos	1
2.1 Objetivo general	1
2.2 Objetivos específicos	1
2.3 Problemática identificada	2
3. Declaración de alcance (Project Scope Statement)	2
3.1 Objetivos del negocio	2
3.2 Límites del proyecto	2
3.3 Entregables principales	3
4. Metodología de trabajo	4
4.1 Herramienta de gestión	4
4.2 Product Backlog — Historias de usuario (Sprint 1–2, Avance 1)	4
4.3 Sprint Planning	5
4.4 Evidencia de ceremonias ágiles	5
5. Arquitectura de la solución	6
5.1 Caso de uso empresarial — USIL Paraguay	6
5.2 Roles dentro del equipo de desarrollo	6
6. Modelo de datos (MER / DER)	7
6.1 Diccionario de datos	8
6.2 Normalización — Justificación en 3FN	11
7. Código fuente / Dockerfiles — Entorno base	11
7.1 Estructura del proyecto	11
7.2 docker-compose.yml	11
7.3 docker/php/Dockerfile	12
7.4 Variables de entorno (.env.example)	13
7.5 Puesta en marcha local	13
8. Calendario de entregas	13
9. Conclusiones	14



1. Introducción

El presente informe corresponde al Avance 1 del Proyecto Integrador de la asignatura Programación IV, y documenta la etapa de planificación de una solución web empresarial orientada a resolver una problemática real de una institución localizada en Paraguay.

A diferencia de los rubros PyME sugeridos por la cátedra (retail, logística, gastronomía, fintech), el equipo optó por adaptar el caso a una institución con una problemática operativa concreta y ya identificada: la Universidad San Ignacio de Loyola, Sede Paraguay (USIL Paraguay), y su necesidad de sincronizar dos plataformas académicas que hoy operan de forma aislada: Canvas LMS y Microsoft Teams.

El proyecto se encuentra en etapa de planificación: el desarrollo del código se iniciará en el Sprint 1, conforme al cronograma detallado en la sección 4.3 de este documento. Por tratarse de un entregable de planificación, este informe no incluye evidencia de pruebas en navegador ni código fuente ejecutado — dichos entregables corresponden a los Avances 2, 3 y 4 según el calendario oficial de la cátedra (sección 8).

2. Objetivos

2.1 Objetivo general

Diseñar y planificar, para su posterior desarrollo, una plataforma web de administración que centralice y automatice la sincronización de usuarios, cursos y equipos entre Canvas LMS y Microsoft Teams para USIL Paraguay, reduciendo el trabajo manual del personal de TI y garantizando consistencia entre ambas plataformas, con trazabilidad completa de cada operación.

2.2 Objetivos específicos

- Centralizar en un único panel administrativo la gestión de altas, matrículas y bajas que hoy se realizan por separado en Canvas y en Microsoft 365.
- Automatizar la creación de equipos de Microsoft Teams a partir de los cursos activos en Canvas, evitando la creación manual y la duplicación de nombres.
- Registrar en un log de auditoría toda operación de sincronización (usuario, curso, tipo de operación, resultado), para trazabilidad ante incidentes.
- Exponer una API REST documentada que permita a futuro integrar la plataforma con otros sistemas institucionales (p. ej. el sistema de gestión académica de USIL).
- Desplegar la solución en contenedores gestionados con Portainer y balanceados con HAProxy, cumpliendo los requisitos de infraestructura de la asignatura.



2.3 Problemática identificada

USIL Paraguay administra dos plataformas independientes para su operación académica: Canvas LMS (gestión de cursos, matrículas y calificaciones) y Microsoft 365 / Teams (identidad institucional, correo y espacios de colaboración por curso). Actualmente estas dos plataformas no están integradas entre sí, lo que obliga al personal de TI a ejecutar manualmente, curso por curso y usuario por usuario, tareas repetitivas de:

- Alta de usuarios (estudiantes y docentes) en Canvas LMS y en Azure Active Directory / Microsoft 365.
- Creación del equipo de Microsoft Teams correspondiente a cada curso o sección publicada en Canvas.
- Matriculación de usuarios en el curso de Canvas y, en paralelo, su alta como miembro del Team asociado.
- Baja o desvinculación de usuarios en ambas plataformas al finalizar un período académico o egreso.
- Trazabilidad y auditoría de estas operaciones, hoy dispersa entre correos, hojas de cálculo y el historial nativo de cada plataforma.

Esta falta de integración genera inconsistencias (usuarios matriculados en Canvas sin acceso al Team del curso, o viceversa), demoras operativas y ausencia de un registro centralizado de auditoría — la problemática real que este proyecto busca resolver.

3. Declaración de alcance (Project Scope Statement)

3.1 Objetivos del negocio

La solución impacta directamente en la operación diaria del área de TI de USIL Paraguay: reduce el tiempo dedicado a tareas repetitivas de alta/baja de usuarios y creación de equipos (hoy realizadas a mano, curso por curso), disminuye el riesgo de inconsistencias entre Canvas y Teams, y deja un registro auditable de cada operación — algo que hoy no existe de forma centralizada.

3.2 Límites del proyecto

Incluido en este semestre (In-Scope):

- Módulo de autenticación restringido a personal administrativo de TI.
- Listado y filtro de cursos sincronizados desde Canvas (consulta vía API REST de Canvas).
- Creación de equipos de Microsoft Teams asociados a un curso (vía Microsoft Graph API).
- Matriculación combinada de un usuario en curso (Canvas) + Team (Teams) desde una sola acción.
- Baja combinada de usuario en ambas plataformas.



- Registro de auditoría de las operaciones anteriores.
- API REST documentada para las operaciones anteriores.
- Despliegue en contenedores Docker gestionados con Portainer.
- Balanceo de carga con HAProxy entre dos arquitecturas de despliegue (Arquitectura A y B) — comparativa pendiente de definición formal por la cátedra.

Fuera de alcance este semestre (Out-of-Scope):

- Sincronización de calificaciones o contenido académico entre Canvas y Teams.
- Autoservicio para que estudiantes o docentes se auto-matriculen (el flujo es siempre operado por TI).
- Integración con el sistema de gestión académica interno de USIL (SIS) más allá de lectura de código SIS del curso.
- Aplicación móvil nativa.
- Facturación o módulos administrativos ajenos a la sincronización Canvas–Teams.

Esta separación explícita busca evitar la corrupción de alcance (scope creep) durante el semestre, dado el volumen de requisitos de infraestructura (Docker, Portainer, HAProxy) que ya exige la cátedra.

3.3 Entregables principales

Entregable	Descripción
Módulos funcionales	Autenticación, gestión de cursos sincronizados, matriculación combinada, bajas combinadas, auditoría.
API REST	Endpoints documentados (OpenAPI/Swagger) para las operaciones de sincronización.
Despliegue en Portainer	Stack Docker Compose (Laravel + MySQL + Redis + Nginx) gestionado desde Portainer.
Balanceo con HAProxy	Distribución de tráfico entre Arquitectura A y B, con health checks y failover.
Documentación	Este documento (Alcance, MER, metodología) más documentación técnica en el repositorio Git.

4. Metodología de trabajo

El equipo adopta Scrum como metodología ágil de gestión del proyecto durante el semestre, por ajustarse naturalmente al calendario de hitos definido por la cátedra (Avances 1 a 4), donde cada hito agrupa entre 2 y 3 Sprints de 2 semanas.

4.1 Herramienta de gestión

Se utiliza Trello como tablero ágil digital, organizado en un tablero único “Sync USIL — Canvas ↔ Teams” con las siguientes listas:

Lista	Propósito
Product Backlog	Historias de usuario priorizadas, aún no asignadas a un Sprint.
Sprint Backlog	Historias comprometidas para el Sprint en curso.
En progreso	Tareas que un integrante está desarrollando activamente.
En revisión / QA	Tareas terminadas a la espera de revisión de código o prueba funcional.
Hecho	Tareas completadas, validadas y con evidencia adjunta (captura, enlace a commit).

4.2 Product Backlog — Historias de usuario (Sprint 1–2, Avance 1)

Historias priorizadas para los dos primeros Sprints, con formato “Como [rol], quiero [acción], para [beneficio]” y criterios de aceptación.

#	Historia de usuario	Criterios de aceptación	Prioridad
HU-01	Como administrador de TI, quiero iniciar sesión con mi cuenta institucional, para acceder al panel de forma segura.	• Login solo habilitado para correos del dominio institucional. • Sesión expira tras inactividad.	Alta
HU-02	Como administrador de TI, quiero ver el listado de cursos sincronizados desde Canvas, para saber cuáles ya tienen Team creado.	• Se muestra nombre, código SIS, período y estado de sincronización. • Filtro por período académico.	Alta
HU-03	Como administrador de TI, quiero crear un Team de Microsoft Teams a partir de un curso de Canvas con un clic, para no crearlo manualmente.	• El Team hereda nombre y código del curso. • Se registra el teams_group_id resultante.	Alta
HU-04	Como administrador de TI, quiero matricular un usuario en un curso y agregarlo automáticamente al Team asociado, para evitar hacerlo dos veces.	• Falla controlada si el usuario ya está matriculado (sin duplicar). • Se refleja en ambas plataformas.	Alta
HU-05	Como administrador de TI, quiero consultar un historial de auditoría de sincronizaciones, para investigar incidentes.	• Registro con usuario, acción, entidad afectada y fecha. • Búsqueda por usuario o curso.	Media



#	Historia de usuario	Criterios de aceptación	Prioridad
HU-06	Como administrador de TI, quiero dar de baja a un usuario en Canvas y Teams desde una sola pantalla, para agilizar el proceso de egreso.	• Confirmación previa antes de ejecutar. • Resultado visible por plataforma (Canvas / Teams).	Media

4.3 Sprint Planning

Sprint	Duración	Objetivo del Sprint	Historias incluidas
Sprint 1	2 semanas	Autenticación y visualización de cursos sincronizados.	HU-01, HU-02
Sprint 2	2 semanas	Creación de Teams y matriculación combinada.	HU-03, HU-04
Sprint 3	2 semanas	Auditoría y bajas combinadas (previsto para Avance 2).	HU-05, HU-06

4.4 Evidencia de ceremonias ágiles

Se mantendrá un registro breve (acta de una página) de cada ceremonia en la carpeta /docs/ceremonias del repositorio:

- Daily Standup: registro semanal resumido (no diario) dado el formato de la cátedra, con avances, bloqueos y próximos pasos por integrante.
- Sprint Review: demostración de las historias completadas al cierre de cada Sprint, con captura de pantalla o grabación corta.
- Sprint Retrospective: 3 columnas (Qué funcionó / Qué mejorar / Acciones) al cierre de cada hito de Avance.

5. Arquitectura de la solución

5.1 Caso de uso empresarial — USIL Paraguay

Rubro: Servicios educativos / Educación superior — gestión de identidad y colaboración académica.

Actor	Interacción con el sistema
Administrador de TI (USIL)	Usuario principal del sistema: ejecuta altas, matriculaciones, bajas y consulta auditoría.
Canvas LMS	Sistema externo consultado vía API REST para leer cursos, usuarios y matrículas.
Microsoft Graph / Teams	Sistema externo invocado vía API para crear equipos y gestionar membresías.
Docente / Estudiante	Beneficiario indirecto: recibe acceso consistente a curso (Canvas) y Team (Teams) sin gestión manual duplicada.

Flujo principal (caso de uso: matriculación combinada)

- 1. El administrador de TI busca un curso sincronizado desde Canvas en el panel.
- 2. Si el curso aún no tiene Team asociado, el sistema lo crea vía Microsoft Graph API y guarda el teams_group_id.
- 3. El administrador busca al usuario a matricular (por email institucional o cédula).
- 4. El sistema valida que el usuario no esté ya matriculado en el curso (evita duplicados).
- 5. El sistema matricula al usuario en Canvas y lo agrega como miembro del Team correspondiente.
- 6. El sistema registra la operación en el log de auditoría con resultado (éxito / error por plataforma).

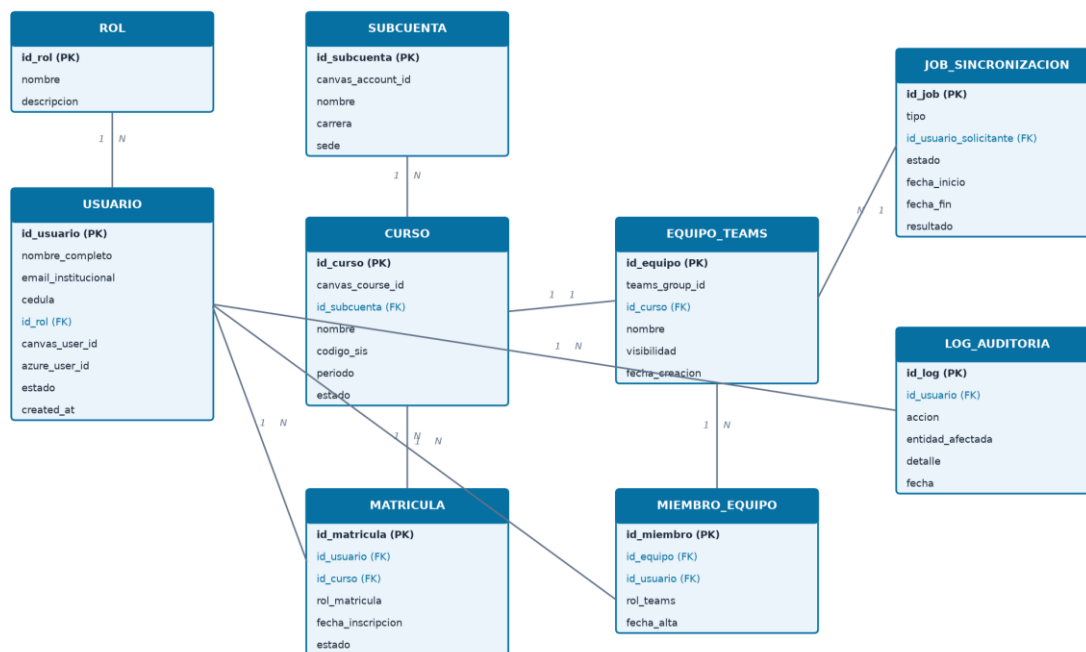
5.2 Roles dentro del equipo de desarrollo

Rol	Responsabilidad
Scrum Master / Project Manager	Metodología ágil, gestión del tablero Trello, control del alcance y remoción de bloqueos del equipo.
Backend & Software Architect Lead	Lógica de negocio en Laravel, arquitectura de datos, migraciones, Eloquent y diseño de la API REST.
DevOps & Cloud Engineer	Dockerfile, docker-compose.yml, stack en Portainer y configuración (.cfg) de HAProxy.
Frontend & QA Engineer	Vistas del panel administrativo, consumo de los endpoints de la API, pruebas funcionales y benchmarking.

6. Modelo de datos (MER / DER)

El modelo de datos representa el dominio de sincronización entre Canvas LMS y Microsoft Teams: usuarios institucionales, su rol, los cursos y subcuentas de Canvas, las matrículas, los equipos de Teams y sus miembros, y los registros operativos (jobs de sincronización y auditoría).

Modelo Entidad-Relación (MER) — Sistema de Sincronización Canvas LMS ↔ Microsoft Teams (USIL Paraguay)





6.1 Diccionario de datos

ROL

Campo	Tipo	Restricción	Descripción
id_rol	BIGINT	PK, AUTO_INCREMENT	Identificador del rol.
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del rol (Admin TI, Docente, Estudiante).
descripcion	VARCHAR(255)	NULL	Descripción del alcance del rol.

USUARIO

Campo	Tipo	Restricción	Descripción
id_usuario	BIGINT	PK, AUTO_INCREMENT	Identificador interno del usuario.
nombre_completo	VARCHAR(150)	NOT NULL	Nombre y apellido.
email_institucional	VARCHAR(150)	NOT NULL, UNIQUE	Correo @usil.edu.py.
cedula	VARCHAR(15)	NOT NULL, UNIQUE	Cédula de identidad paraguaya.
id_rol	BIGINT	FK → ROL(id_rol)	Rol asignado al usuario.
canvas_user_id	BIGINT	NULL, índice	ID del usuario en Canvas LMS.
azure_user_id	VARCHAR(60)	NULL, índice	Object ID en Azure AD.
estado	ENUM	NOT NULL, DEFAULT 'activo'	activo / inactivo.
created_at	TIMESTAMP	NOT NULL	Fecha de alta en el sistema.

SUBCUENTA

Campo	Tipo	Restricción	Descripción
id_subcuenta	BIGINT	PK, AUTO_INCREMENT	Identificador interno.
canvas_account_id	BIGINT	NOT NULL, UNIQUE	ID de subcuenta en Canvas.
nombre	VARCHAR(120)	NOT NULL	Nombre de la subcuenta / facultad.
carrera	VARCHAR(120)	NULL	Carrera asociada, si aplica.
sede	VARCHAR(60)	NOT NULL	Sede de USIL Paraguay.



CURSO

Campo	Tipo	Restricción	Descripción
id_curso	BIGINT	PK, AUTO_INCREMENT	Identificador interno.
canvas_course_id	BIGINT	NOT NULL, UNIQUE	ID del curso en Canvas.
id_subcuenta	BIGINT	FK → SUBCUENTA(id_subcuenta)	Subcuenta a la que pertenece.
nombre	VARCHAR(150)	NOT NULL	Nombre del curso.
codigo_sis	VARCHAR(30)	NULL, índice	Código SIS del curso.
periodo	VARCHAR(20)	NOT NULL	Período académico (p. ej. 2026-2).
estado	ENUM	NOT NULL	unpublished / available / completed.

MATRICULA

Campo	Tipo	Restricción	Descripción
id_matricula	BIGINT	PK, AUTO_INCREMENT	Identificador interno.
id_usuario	BIGINT	FK → USUARIO(id_usuario)	Usuario matriculado.
id_curso	BIGINT	FK → CURSO(id_curso)	Curso de la matrícula.
rol_matricula	ENUM	NOT NULL	estudiante / docente.
fecha_inscripcion	DATETIME	NOT NULL	Fecha de la matriculación.
estado	ENUM	NOT NULL, DEFAULT 'activa'	activa / retirada.

Restricción de unicidad: UNIQUE (id_usuario, id_curso) — evita matrículas duplicadas del mismo usuario en el mismo curso, requisito explícito del sistema (HU-04).

EQUIPO_TEAMS

Campo	Tipo	Restricción	Descripción
id_equipo	BIGINT	PK, AUTO_INCREMENT	Identificador interno.
teams_group_id	VARCHAR(60)	NOT NULL, UNIQUE	ID del grupo en Microsoft Graph.
id_curso	BIGINT	FK → CURSO(id_curso), UNIQUE	Curso asociado (relación 1:1).
nombre	VARCHAR(150)	NOT NULL	Nombre del Team.
visibilidad	ENUM	NOT NULL	Private / Public.
fecha_creacion	DATETIME	NOT NULL	Fecha de creación del Team.



MIEMBRO_EQUIPO

Campo	Tipo	Restricción	Descripción
id_miembro	BIGINT	PK, AUTO_INCREMENT	Identificador interno.
id_equipo	BIGINT	FK → EQUIPO_TEAMS(id_equipo)	Team al que pertenece.
id_usuario	BIGINT	FK → USUARIO(id_usuario)	Usuario miembro.
rol_teams	ENUM	NOT NULL	Owner / Member.
fecha_alta	DATETIME	NOT NULL	Fecha de alta como miembro.

JOB_SINCRONIZACION

Campo	Tipo	Restricción	Descripción
id_job	BIGINT	PK, AUTO_INCREMENT	Identificador interno.
tipo	ENUM	NOT NULL	alta_usuario / crear_team / matricula / baja.
id_usuario_solicitante	BIGINT	FK → USUARIO(id_usuario)	Administrador que ejecutó la acción.
estado	ENUM	NOT NULL	pendiente / en_proceso / completado / error.
fecha_inicio	DATETIME	NOT NULL	Inicio de ejecución del job.
fecha_fin	DATETIME	NULL	Fin de ejecución del job.
resultado	TEXT	NULL	Detalle del resultado o error.

LOG_AUDITORIA

Campo	Tipo	Restricción	Descripción
id_log	BIGINT	PK, AUTO_INCREMENT	Identificador interno.
id_usuario	BIGINT	FK → USUARIO(id_usuario)	Usuario que ejecutó la acción.
accion	VARCHAR(80)	NOT NULL	Acción realizada (p. ej. matricula_creada).
entidad_afectada	VARCHAR(80)	NOT NULL	Entidad y ID afectados.
detalle	TEXT	NULL	Detalle adicional en formato JSON.
fecha	TIMESTAMP	NOT NULL	Fecha y hora del evento.

6.2 Normalización — Justificación en 3FN

El modelo cumple con la Tercera Forma Normal (3FN):

- 1FN: todos los atributos son atómicos (p. ej. nombre_completo no se descompone porque se usa como unidad; no existen campos multivaluados ni listas dentro de una columna).
- 2FN: no hay dependencias parciales — todas las tablas usan una clave primaria simple (id autonumérico), por lo que no existen atributos que dependan solo de una parte de una clave compuesta.
- 3FN: no hay dependencias transitivas — por ejemplo, la carrera y sede de un curso no se repiten en CURSO, sino que se obtienen a través de SUBCUENTA (id_subcuenta), evitando redundancia y anomalías de actualización.

Las tablas MATRICULA y MIEMBRO_EQUIPO son tablas asociativas que resuelven relaciones N:M (usuarios $\leftrightarrow$ cursos, equipos $\leftrightarrow$ usuarios) y agregan atributos propios de la relación (rol_matricula, fecha_inscripcion, rol_teams), consistente con el diseño relacional estándar.

7. Código fuente / Dockerfiles — Entorno base

Se define el entorno de contenedores para desarrollo local, previo a la integración de Portainer y HAProxy (previstas para los Avances 2 y 3). El stack contempla cuatro servicios: aplicación Laravel (PHP-FPM), servidor web Nginx, base de datos MySQL y Redis para sesiones centralizadas.

7.1 Estructura del proyecto

```
sync-usil-canvas-teams/
├── app/                # Código Laravel (Models, Controllers, Services)
├── docker/
│   ├── nginx/default.conf
│   └── php/Dockerfile
├── docker-compose.yml
├── .env.example
└── database/migrations/ # Migraciones del MER (sección 6)
```

7.2 docker-compose.yml

```
version: '3.9'
services:
  app:
    build:
      context: .
      dockerfile: docker/php/Dockerfile
    container_name: sync_usil_app
    volumes:
      - ./:/var/www/html
    environment:
      - DB_HOST=db
      - DB_DATABASE=sync_usil
      - DB_USERNAME=sync_usil
      - DB_PASSWORD=${DB_PASSWORD}
      - REDIS_HOST=redis
    depends_on:
```



```
- db
- redis
networks: [sync_net]

webserver:
  image: nginx:1.27-alpine
  container_name: sync_usil_web
  ports:
    - '8080:80'
  volumes:
    - ./var/www/html
    - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf
  depends_on: [app]
  networks: [sync_net]

db:
  image: mysql:8.0
  container_name: sync_usil_db
  environment:
    - MYSQL_DATABASE=sync_usil
    - MYSQL_USER=sync_usil
    - MYSQL_PASSWORD=${DB_PASSWORD}
    - MYSQL_ROOT_PASSWORD=${DB_ROOT_PASSWORD}
  volumes:
    - db_data:/var/lib/mysql
  ports:
    - '3306:3306'
  networks: [sync_net]

redis:
  image: redis:7-alpine
  container_name: sync_usil_redis
  networks: [sync_net]

volumes:
  db_data:

networks:
  sync_net:
    driver: bridge
```

7.3 docker/php/Dockerfile

```
FROM php:8.3-fpm

RUN apt-get update && apt-get install -y \
  git unzip libzip-dev libpng-dev \
  && docker-php-ext-install pdo_mysql zip gd

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html
COPY . .
RUN composer install --no-dev --optimize-autoloader

EXPOSE 9000
CMD ["php-fpm"]
```

7.4 Variables de entorno (.env.example)

```
APP_NAME="Sync USIL Canvas-Teams"
APP_ENV=local
APP_URL=http://localhost:8080

DB_CONNECTION=mysql
DB_HOST=db
DB_PORT=3306
DB_DATABASE=sync_usil
DB_USERNAME=sync_usil
DB_PASSWORD=changeme

SESSION_DRIVER=redis
REDIS_HOST=redis

CANVAS_BASE_URL=https://usilparaguay.instructure.com
CANVAS_ACCESS_TOKEN=
AZURE_TENANT_ID=
AZURE_CLIENT_ID=
AZURE_CLIENT_SECRET=
```

7.5 Puesta en marcha local

```
$ cp .env.example .env
$ docker compose up -d --build
$ docker compose exec app php artisan key:generate
$ docker compose exec app php artisan migrate
```

Este entorno se documenta a nivel de definición (Avance 1); su despliegue mediante Portainer y el balanceo con HAProxy entre dos arquitecturas activas se abordan en los Avances 2 y 3 respectivamente, conforme al calendario de la asignatura.

8. Calendario de entregas

Hito	Fecha límite	Entregables
Avance 1	28 de agosto	Alcance + caso institucional · Metodología (Trello + Product Backlog) · MER/DER + diccionario de datos · Entorno base Docker.
Avance 2	25 de septiembre	Reporte de Sprints · Módulo CRUD principal funcional · Sesiones centralizadas (Redis) · Despliegue inicial en Portainer.
Avance 3	30 de octubre	Evidencia de ceremonias ágiles · Integración de HAProxy · Arquitecturas A y B activas · Demostración de failover.
Avance 4	27 de noviembre	Cierre metodológico · Benchmarking RPS/latencia A vs B · Dashboard HAProxy Stats con tráfico real · Documentación técnica completa.



9. Conclusiones

La etapa de planificación del proyecto permitió identificar con precisión una problemática real y vigente dentro de USIL Paraguay: la ausencia de integración entre Canvas LMS y Microsoft Teams obliga a un trabajo manual, repetitivo y propenso a inconsistencias por parte del personal de TI. El caso institucional elegido, al estar basado en un proceso operativo real, aporta un alcance concreto y verificable frente a un caso PyME hipotético.

La adopción de Scrum como metodología, junto con un Product Backlog acotado a seis historias de usuario para los dos primeros Sprints, deja una base clara y priorizada para iniciar el desarrollo. El modelo de datos diseñado (nueve entidades en Tercera Forma Normal) cubre el dominio completo del problema — usuarios, cursos, matrículas, equipos de Teams y trazabilidad — sin incorporar funcionalidades fuera del alcance definido.

El entorno Docker documentado (Laravel + Nginx + MySQL + Redis) establece la base de infraestructura sobre la cual se construirán, en los próximos avances, el despliegue en Portainer y el balanceo de carga con HAProxy exigidos por la cátedra. Con esta planificación completa, el equipo queda en condiciones de iniciar el Sprint 1 según el cronograma establecido.